

AI IN ACTION · NGÀY 5

Thiết kế sản phẩm AI cho sự không chắc chắn

Từ khả năng của model đến sản phẩm người dùng tin tưởng

VinUniversity · Day 5 · 2026



[Tên giảng viên]

[Chức danh / Vai trò]

- [Kinh nghiệm 1]
- [Kinh nghiệm 2]
- [Kinh nghiệm 3]
- [Kinh nghiệm 4]

Focus: [Lĩnh vực chuyên môn]

Mục tiêu hôm nay

3 câu hỏi xuyên suốt ngày:

- 1. Agent chạy được — có đang build thành product không?**
- 2. Requirement / UX / eval cho AI khác gì?**
- 3. Làm sao gộp thành SPEC + hackathon?**

Flow buổi học

SÁNG — LECTURE + PRACTICE

- Provocation: agent chạy ≠ product
- AI = probabilistic → Auto/Aug
- 3 trụ: Requirement · UX · Eval
- Canvas preview + mini practice

CHIỀU — LECTURE + HACKATHON

- Feedback + Data Flywheel
- ROI 3 kịch bản + Q&A
- Hackathon briefing 16:00
- Chia nhóm + chọn track + draft SPEC

SẢN PHẨM CẦN NỘP

➤ Topic + SPEC draft

Submit: midnight

➤ Prototype

Day 6, 16:00

**Agent của bạn chạy được rồi.
Vậy tại sao có thể không ai dùng?**

3 cases: agent chạy ≠ product thành công

Trường hợp	Vấn đề	Kết quả
Google AI Overview	Rắc "fairy dust" AI lên mọi sản phẩm	User không tin, không dùng
Chegg	AI thay thế core value	−90% stock
Agent của bạn	Chưa chứng minh user cần	?

Pattern

0 → 80% = 1 tuần. 80 → 95% = 4x effort. Vấn đề không phải "AI có chạy không" mà "chạy đủ tốt để user tin không".

Vấn đề không phải AI yếu. Vấn đề là ta đang đối xử AI như phần mềm thường.

2012–2020 · Siri, Alexa, Google Now

Interface quá tham vọng (voice, proactive)

AI chưa đủ giỏi

→ User thất vọng, bỏ dùng. Alexa chỉ giỏi hẹn giờ và mở nhạc.

2023–nay · ChatGPT, Claude, Copilot

AI cực kỳ giỏi

Interface vẫn chỉ là chat box

→ User không khai thác hết capability.

“AI có trí tuệ đáng kinh ngạc, nhưng giao diện vẫn chỉ như chatbot modem quay số AOL.”

— Aparna Chennapragada, CPO Microsoft (Lenny's Podcast)

Activity nhanh: 5 câu hỏi

1. Ai dùng?
2. Khi đúng → user được gì?
3. Khi sai → chuyện gì?
4. Bao nhiêu % đúng là đủ?
5. Mỗi lần chạy tốn gì?

Nhóm · 5 phút

AI = xác suất + cách deploy

Phần mềm truyền thống dự đoán hành vi. AI dự đoán xác suất.

01

Phần mềm truyền thống vs AI product

	Truyền thống	AI
Kết quả	Chính xác 100%	Xác suất + sai số
Ví dụ	"User ở công viên?" = check GPS	"Ảnh có chim?" = 87% độ tin cậy
Kiểm thử	Đạt / không đạt	Phân bố chất lượng
Lỗi	Bug — sửa được	Sai xác suất — quản lý được

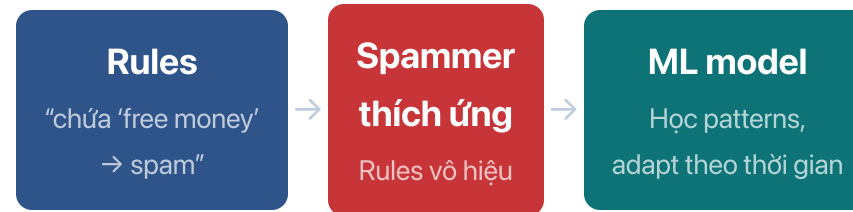
"Bạn không biết user sẽ tương tác thế nào, và cũng không biết LLM sẽ phản hồi ra sao. Cả ba — input, output, process — đều không chắc chắn."

Takeaway

Nếu sản phẩm dùng AI, bạn đang thiết kế cho **uncertainty**.

Ví dụ: Spam filter

Từ rules → ML → trade-off



Trade-off của ML: ML = xác suất → false positive (email quan trọng bị xóa), false negative (spam lọt). Không có "đúng 100%".

→ Nếu AI = xác suất, thì 3 thứ phải thay đổi...

Nếu AI = xác suất → 3 thứ phải thay đổi

AI = probabilistic →

1. Requirement khác

- Spec ngưỡng + failure behavior
- Không spec exact behavior
- 4-path user stories

2. UX khác

- Hệ thống xử lý khi AI sai
- Không phải trang trí
- Design cho lúc sai, không chỉ lúc đúng

3. Eval khác

- Đo phân bố chất lượng
- Không pass/fail
- PM define "good enough"

Cách bạn deploy AI thay đổi mọi thứ.

Ví dụ: Augmentation vs Automation

Augmentation — GitHub Copilot

30% accuracy

→ 20M users

- Ghost text — reject = 0 friction
- Suggest-only, không auto-action
- User luôn có quyền kiểm soát

Automation — Spam filter

User không thấy email bị xóa nhầm

- Precision cực cao bắt buộc
- False positive = mất email quan trọng
- User không có cơ hội sửa

Tại sao cùng AI mà khác nhau đến vậy? →

Automation vs Augmentation

	Automation	Augmentation
AI làm gì	Tự động hoàn toàn, user không thấy	Gợi ý, user quyết định
Khi AI sai	User không biết (nguy hiểm)	User thấy và sửa
Yêu cầu	Precision cực cao, fallback tự động	Ngưỡng vừa đủ, user là fallback
UX	Ẩn, bảng theo dõi	Gợi ý / ghost text / bảng bên
Đánh giá	Phải đo AI sai bao nhiêu — vì user không thấy	Đo user có dùng không — vì user tự lọc
Feedback	Tín hiệu ngầm (bỏ qua, ghi đè)	Phản hồi trực tiếp + sửa lỗi
Rủi ro tin tưởng	Tin sai (user buông tay vô lăng)	Không tin (user bỏ qua gợi ý)

“Nếu model đúng 60%, bạn sẽ phải build product theo cách hoàn toàn khác.” — Kevin Weil, CPO OpenAI

Cùng 1 bài toán — AI email assistant:

60%	Copilot	User review mọi email trước khi gửi
95%	Semi-auto	Auto-send routine, flag email quan trọng
99.5%	Autopilot	User chỉ set rules, AI xử lý hết

→ Accuracy quyết định auto hay aug.

Trong thực tế: bắt đầu aug, tăng dần auto

Ví dụ: agent hỗ trợ khách hàng

V1: Routing	V2: Copilot	V3: Tự động
 AI làm: Phân loại + chuyển ticket đúng chỗ  Người làm: Xử lý 100% ticket Thu: data phân loại, cấu trúc prompt	 AI làm: Gợi ý draft trả lời  Người làm: Review + sửa draft Thu: % draft dùng nguyên? Phần nào bị sửa? = eval miễn phí	 AI làm: Tự giải quyết ticket  Người làm: Chỉ xem edge cases Thu: vòng lặp tự động hoàn toàn

Mỗi version thu data để quyết định lên version tiếp → không đoán mò

 **Nhảy thẳng V3:** “Chúng tôi phải đóng product vì quá nhiều lỗi, không đếm hết các vấn đề mới phát sinh.”

3 trụ: Requirement · UX · Eval

Giảng + Discord exercises + UX workshop

02

Block 2: cách chúng ta làm

1 — Requirement
Giảng framework
+ Discord: tìm lỗi spec
~15 phút

2 — Eval
Giảng framework
+ Discord: P hay R?
~15 phút

3 — UX Workshop
Phân tích sản phẩm AI thật
Sketch as-is / to-be
~40 phút

4 — Debrief
Kết nối 3 trụ
→ Canvas preview
~15 phút

Trụ	Hình thức	Tham khảo
Requirement	Giảng + Discord “tìm lỗi spec”	
Eval	Giảng + Discord “precision hay recall?”	
UX	Workshop: phân tích sản phẩm AI thật (nhóm, 40 phút)	

Requirement: AI spec $\neq$ exact behavior

Requirement truyền thống

- “Click X $\rightarrow$ show Y”
- Feature list
- 1 user story = 1 path

AI Requirement

- “Hỏi X $\rightarrow$ Y $\sim$ 85%, confidence $<$ 60% $\rightarrow$ fallback”
- Failure mode library
- 4-path user stories

Luận điểm

AI spec = định nghĩa kết quả, không phải hành vi chính xác. Failure mode library thay vì feature list.

Descript — pricing tier gắn với AI quality: tier thấp nhận accuracy thấp hơn, tier cao nhận tốt hơn. Requirement phải gắn chất lượng AI vào business model, không chỉ “AI hoạt động tốt”.

Granola — dùng calendar context gợi ý nội dung hợp. Requirement = “helpful suggestion”, không spec exact output.

Chỉ define outcome + ngưỡng chấp nhận được — không phải behavior cụ thể.

Thư viện các dạng lỗi (failure mode library)

Khi nào xảy ra	Hậu quả	Giải pháp
Input ngoài phạm vi	Bị thông tin	Phát hiện + “Tôi không biết”
Câu hỏi mơ hồ	Trả lời sai ngữ cảnh	Hỏi lại / làm rõ
Dữ liệu cũ / thiếu	Kết quả lỗi thời	Gắn thời gian + cảnh báo

Câu hỏi then chốt

Top 3 cách product có thể fail?

Discord: tìm lỗi trong spec

~3 phút gõ — chọn 1 đề

Đề 1 — Chatbot CSKH ngân hàng

"Chatbot trả lời câu hỏi khách hàng về tài khoản, thẻ, vay. Hiểu tiếng Việt có dấu và không dấu. Phản hồi trong 3 giây. Hỗ trợ 24/7."

Đề 2 — AI tóm tắt meeting

"AI tự động tóm tắt cuộc họp thành bullet points. Bắt action items, người chịu trách nhiệm, và tự động gửi email tới người đó."

Đề 3 — AI chấm bài luận

"AI chấm điểm bài luận tiếng Việt của học sinh. Cho điểm từ 1-10 kèm nhận xét ngắn. Chấm xong trong 5 phút. Hỗ trợ mọi chủ đề."

Đề 4 — AI lọc CV ứng viên

"AI đọc CV ứng viên và xếp hạng phù hợp cho vị trí. Lọc top 20 CV từ 500 hồ sơ. Xử lý trong 1 giờ."

Gõ vào Discord

Bạn là engineer nhận spec này. Build được không? Thiếu gì?

Gõ 1-2 thứ thiếu.

Eval: quality = phân bố, không phải yes/no

Test truyền thống

— Pass / fail

— QA owns

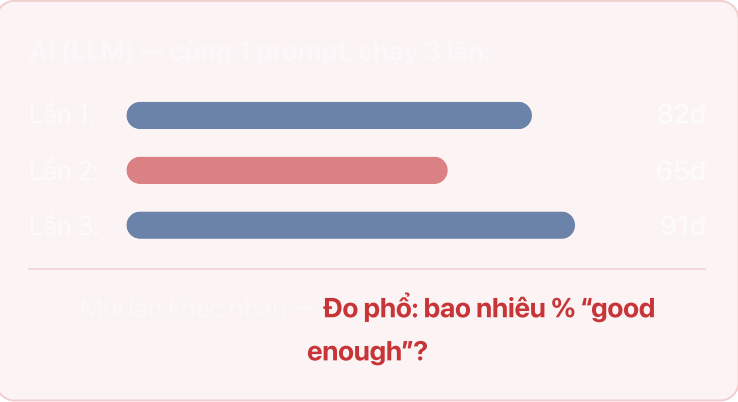
— Bug = fix

AI Eval

— Đo phân bố chất lượng

— PM define “good enough”

— PM chọn precision hay recall



1

2

3

Copilot — chọn recall: gợi ý nhiều, user tự filter.
30% accuracy tạo value vì cost of reject = 0 (ghost text biến mất).

Harvey (legal AI) — chọn precision: sai = thiệt hại pháp lý.
Thả bỏ lot case còn hơn cite sai precedent. 1 lần sai = mất uy tín

Spam filter — FP = email quan trọng bị xóa (user không biết).
FN = spam lọt inbox (user thấy tự xóa). FP tệ hơn → chọn precision.

Precision vs Recall — hiểu qua ví dụ

Precision — “Khi AI nói CÓ, đúng bao nhiêu?”

Trong số những cái AI đánh dấu, bao nhiêu % thật sự đúng?

Precision cao = ít sai nhầm, có thể bỏ sót

Lỗi: false positive (báo động nhầm)

Recall — “AI tìm được bao nhiêu?”

Trong số những cái đúng thật sự, bao nhiêu % AI tìm ra?

Recall cao = ít bỏ sót, có thể báo nhầm

Lỗi: false negative (bỏ sót)

Ví dụ: AI lọc nội dung trẻ em — 100 videos, 10 cái xấu thật sự

	AI đánh dấu “xấu”	AI cho qua
Xấu thật	8 ✓	2 × bỏ sót (FN)
Lành thật	5 × báo nhầm (FP)	85 ✓

Precision = 8/13 = 62%

Trong 13 cái AI đánh dấu xấu, chỉ 8 cái xấu thật

Recall = 8/10 = 80%

Trong 10 cái xấu thật, AI tìm được 8

Takeaway

2 cái bỏ sót = trẻ thấy nội dung xấu → cần **recall cao**.

	Precision	Recall
Câu hỏi	AI sai nhầm nhiều không?	AI bỏ sót nhiều không?
Ưu tiên khi	Sai nhầm = thiệt hại lớn	Bỏ sót = thiệt hại lớn
Ví dụ	Duyệt vay: sai = mất tiền	X-quang: sót = chết người

Precision hay Recall? — phụ thuộc context

	User THẤY & sửa được	User KHÔNG thấy
Sai mà act = hậu quả nặng (FP tệ hơn)	PRECISION VD: Legal RAG, Bank chatbot User thấy những hậu quả nặng	PRECISION VD: Spam filter, Auto-send email Sai mà không ai biết = nguy hiểm
Bỏ lọt = mất giá trị (FN tệ hơn)	RECALL VD: Copilot, FAQ chatbot Gợi ý nhiều, user tự filter	RECALL VD: Content mod (trẻ em), Fraud Bỏ lọt = thảm họa, dù user không thấy

Rule of thumb

Sai mà user KHÔNG BIẾT → thường precision. Nhưng nếu bỏ lọt = thảm họa → recall bất kể user thấy hay không.

Discord: precision hay recall?

~3 phút gõ — tất cả cùng làm

#	Sản phẩm
1	AI lọc nội dung cho app trẻ em
2	Code autocomplete (kiểu Copilot)
3	AI đọc X-quang phát hiện ung thư
4	AI duyệt khoản vay ngân hàng
5	AI gợi ý bài hát (Spotify)

Gõ vào Discord

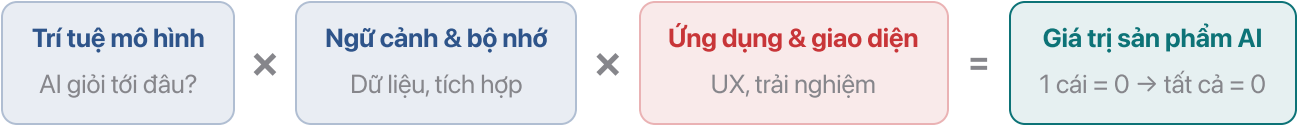
Mỗi sản phẩm nên ưu tiên **precision** (ít sai) hay **recall** (ít bỏ sót)?

Format: số-P hoặc R — lý do 1 câu

Ví dụ: 1-P — vì sai thì user không biết để sửa

UX cho AI = thiết kế cho lúc sai

UX truyền thống	AI UX
— Giao diện, thẩm mỹ — Thiết kế cho lúc đúng — Trang trí trải nghiệm	— Hệ thống xử lý khi AI sai — Thiết kế cho lúc sai — Graceful failure + trust recovery



“Khoảng cách giữa năng lực thật của AI và cách người dùng sử dụng hàng ngày — rất lớn. Đây là bài toán UX, không phải bài toán AI.”
— Mike Krieger, CPO Anthropic (Lenny's Podcast)

- 4 câu UX phải trả lời:**
- (1) Khi đúng → user thấy gì?
 - (2) Khi không chắc → hệ thống làm gì?
 - (3) Khi sai → user sửa thế nào?
 - (4) Khi mất tin → gỡ thế nào?

Xử lý lỗi nhẹ nhàng + niềm tin

Xử lý lỗi nhẹ nhàng

- Nhiều đường (không chỉ 1 kết quả)
- Cho phép sửa (user chỉnh được)
- Thông báo đã sửa (hệ thống ghi nhận)

Niềm tin

- Giải thích lý do (tại sao đưa ra kết quả)
- Hiển thị độ chắc (confidence)
- Đưa ra lựa chọn khác

Copilot ghost-text — 30% accuracy, 20M users hài lòng.

Suggest, không decide. Tiếp tục gõ = ghost text tự biến mất. Cost of reject = 0.

Microsoft Tay — không có failure design → bị manipulate thành racist bot trong vài giờ.

Không safety net, không correction path, không opt-out.

Grammarly — giải thích tại sao gợi ý: "Wordiness — thử 'thrilled' thay vì 'very happy'".

Explain why → user tin và học theo.

Kayak — hiển thị confidence: "Advice: BUY — Confidence: 79%".

User thấy độ chắc chắn → tự quyết định có theo không.

4 thành phần giao diện mới cho AI (Aparna Chennapragada, CPO Microsoft)

Gợi ý ban đầu

Gợi ý cách bắt đầu

VD: ChatGPT starter prompts

Kế hoạch sửa được

User chỉnh sửa kế hoạch

VD: Claude Code hiện plan trước khi code

Cho xem quá trình

Hiển thị quá trình vừa đủ

VD: "đang đọc 15 nguồn..."

Gợi ý tiếp theo

Gợi ý bước tiếp chủ động

VD: Tạo ảnh → "đổi màu?"

"Quá dài dòng = cron job. Quá ngắn gọn = không biết AI đi đúng hướng không." — Aparna, CPO Microsoft (Lenny's Podcast)

Bài tập UX: phân tích sản phẩm AI thật

~40 phút · cá nhân · output: sketch giấy, nộp lấy điểm

Chọn 1 sản phẩm:

MoMo — Trợ thủ AI Moni
Phân loại chi tiêu, chatbot tài chính

15'

Khám phá
Dùng thử app
Marketing vs thực tế?

VN Airlines — Chatbot NEO
Chatbot hỗ trợ KH, tra cứu chuyến bay

10'

Phân tích 4 paths
Đúng / không chắc
Sai / mất tin

V-App — Trợ lý ảo V-AI
Trợ lý voice/text, gợi ý theo context

10'

Sketch to-be
Chọn path yếu nhất
As-is → to-be trên giấy

5'

Share + vote
Chia sẻ trong nhóm
Vote sketch hay nhất

Nộp cá nhân: sketch giấy (as-is + to-be) + ghi chú 4 paths. Điểm cá nhân 10đ. Vote = bonus.
Nice to have: screenshot app + annotate chỗ hay / chỗ gãy.

Debrief: kết nối 3 trụ

Requirement: spec thiếu gì? (Discord A — 4 specs đều thiếu cùng pattern)

Eval: precision hay recall? (Discord C — phụ thuộc context, không phải kỹ thuật)

UX: path nào yếu nhất? (Workshop — sản phẩm thật đều có gap)

3 trụ không độc lập:

Auto/Aug → Requirement khác → UX khác → Eval khác

Đây là vòng tròn, không phải đường thẳng.

→ Tiếp theo: Canvas — gom requirement + UX + eval thành 1 trang product spec.

Nghỉ giải lao — 15 phút

Canvas: gom lại thành lightweight spec

Gộp 3 trụ thành 1 trang product spec

03

AI Product Canvas — 3 cột

VALUE

- User nào? Pain nào?
- Automation hay augmentation?
- Value khi AI đúng?

TRUST

- Precision hay recall?
- Khi sai → user biết/sửa thế nào?
- Trust recovery?

FEASIBILITY

- Cost / latency?
- Dependency / risk chính?

Learning Signal: (1) User correction đi vào đâu? (2) Product thu signal gì để biết đang tốt lên hay tệ đi? (3) Data thuộc loại nào? Có marginal value không?

Instructor demo Canvas live

Điền Canvas cho 1 product mẫu — real-time

Instructor · 10 phút

Nhóm sketch Canvas outline

Sketch Canvas cho project của nhóm

Chưa cần đầy đủ — đánh dấu [?] cho chưa biết

Sketch (15 min) + Peer peek (10 min)

Nhóm · 25 phút

Feedback Loop + Data Flywheel

Sau khi launch, sản phẩm phải học hoặc sẽ xuống cấp.

04

Feedback loop: truyền thống vs AI

	Truyền thống (vật thể)	AI (sinh vật)
Bản chất	Vật thể tĩnh — xuất xong = xong	Sinh vật sống — xuất xong = mới bắt đầu
Phản hồi	Thủ công, theo sprint/quý	Liên tục, tự động
Cải thiện	Team phân tích data → cập nhật	Model tự học từ tương tác
Tài sản riêng	Tính năng, thương hiệu, mạng lưới	Vòng data + model tinh chỉnh



Hiệu ứng Alexa: User hỏi 3 lần, AI sai cả 3 → ngừng dùng vĩnh viễn. Không phản hồi = không cải thiện.

3 loại feedback signal

Ngầm (implicit)

- Thời gian xem, cuộn trang
- Bỏ qua, rời đi
- Chấp nhận / từ chối gợi ý

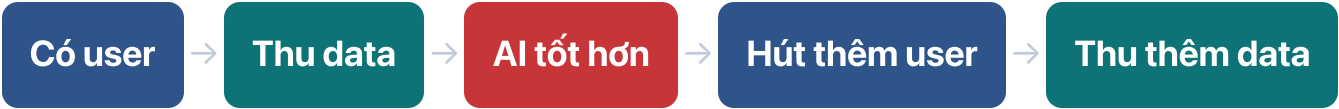
Trực tiếp (explicit)

- Thích / không thích
- Đánh giá tốt/xấu
- Báo cáo / đánh dấu

Sửa lỗi (correction)

- User sửa kết quả
- Đánh dấu sai
- Ghi đè quyết định của AI

Data Flywheel



Ví dụ: MidJourney — user tạo ảnh → rate ảnh → model cải thiện → ảnh đẹp hơn → thêm user.

Ví dụ: Microsoft Dragon — AI ghi chép cho bác sĩ

Giai đoạn 1
Dữ liệu tổng hợp

30–60%

Giai đoạn 2
600K ca thực tế
+ chuyên gia đánh giá

~75%

Giai đoạn 3
Liên tục tối ưu

83% ✓

←←← loop lại: data mới từ bác sĩ dùng hàng ngày → fine-tune tiếp → tốt hơn nữa ←←←

Bài học: Dữ liệu thực + chuyên gia đánh giá >> dữ liệu tổng hợp. Team nhỏ đa chức năng, loop không dừng. — *Asha Sharma, CVP Microsoft (Lenny’s Podcast)*

3 sai lầm: (1) Không thiết kế thu signal từ đầu (2) Thu sai signal (3) Không có pipeline đưa data về model

Takeaway

AI capabilities bị commodity hóa. Chỉ data riêng + cách dùng riêng = lợi thế thật.

Định tính trước, định lượng sau

87% accuracy — nghe ổn.

Nhưng 13% sai ở đâu?

Email FYI phân loại nhầm → phiền. Email urgent phân loại nhầm → mất deal.

4 đòn bẩy để cải tiến (dùng trong hackathon):

1. Sửa **prompt** — rõ hơn, ít ambiguous

2. Sửa **context / KB** — thêm data, bớt noise

3. Sửa **UX** — show confidence, cho user sửa

4. Sửa **evals** — đo đúng thứ cần đo

Hackathon

Ít nhất 1 vòng iterate dùng feedback từ checkpoint. Qualitative trước, quantitative sau.

8 loại dữ liệu giá trị cao

1. Dữ liệu thời gian thực — giá, giao thông, cảm biến

3. Dữ liệu chuyên ngành — y tế, pháp lý, tài chính

5. Học tăng cường (RLHF) — user chọn A vs B

7. Dữ liệu tích hợp — CRM, ERP, API

2. Dữ liệu riêng user — lịch sử, sở thích

4. Dữ liệu đánh giá của người — nhãn, xếp hạng, chú thích

6. Dữ liệu ngữ cảnh — vị trí, thời gian, thiết bị

8. Dữ liệu tổng hợp — tạo tự động, tăng cường

ROI: tính bằng range, không 1 số

AI ROI không cố định — phải tính 3 kịch bản.

05

ROI: truyền thống vs AI

	Truyền thống	AI
ROI	Tính 1 số	Tính 3 kịch bản (conservative / realistic / optimistic)
Mở rộng	Chi phí/đơn vị giảm khi mở rộng	Chi phí/đơn vị có thể tăng theo lượng dùng (suy luận, data, huấn luyện lại)
"Xong"	Ship = xong	Ship = mới bắt đầu (cần maintain + improve)

Chi phí

Suy luận + data + duy trì

Năng lực

Độ chính xác + phạm vi + tốc độ

Tốc độ

Thời gian tạo giá trị + vòng lặp

3 chiến lược ROI

Copilot — Ưu tiên tốc độ

- Giảm thời gian code 55%
- ROI = giờ lập trình viên tiết kiệm
- Chấp nhận độ chính xác vừa đủ

Harvey — Ưu tiên năng lực

- AI cho luật sư
- ROI = vụ mới không thể làm trước
- Precision cực cao bắt buộc

Grammarly — Nhạy chi phí

- 30M+ free users
- ROI = chi phí mỗi lần sửa phải thấp
- Tối ưu chi phí suy luận

ROI cho hackathon SPEC

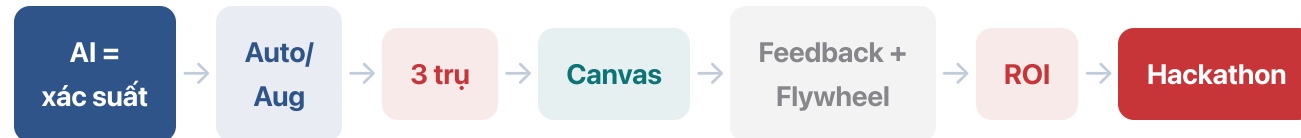
SPEC deliverable phải có ROI 3 kịch bản:

Kịch bản	Giả định	ROI
Thận trọng	Độ chính xác thấp, áp dụng chậm	[Tính cụ thể]
Thực tế	Độ chính xác trung bình, áp dụng vừa	[Tính cụ thể]
Lạc quan	Độ chính xác cao, flywheel hoạt động	[Tính cụ thể]

Lưu ý

Không cần số chính xác. Cần logic rõ ràng và giả định hợp lý.

Tổng kết: chuỗi domino



- 1 AI = xác suất — không phải phần mềm đúng 100%
- 2 Auto vs Aug cascade xuống mọi quyết định
- 3 3 trụ: Requirement cần ngưỡng, UX cho lúc sai, Eval đo phân bố
- 4 Canvas = lightweight spec. Feedback + Flywheel = lợi thế dài hạn
- 5 ROI = 3 kịch bản, không 1 số

Hỏi đáp

Trước khi vào hackathon briefing — còn gì chưa rõ?

Hackathon 2 ngày: từ SPEC đến Demo

Day 5 16:00–18:00 → Day 6 cả ngày

HACK

Timeline hackathon

Thời gian	Hoạt động	Deliverable
Day 5 — 16:00	Briefing + chia nhóm + chọn track + draft SPEC	
Day 5 — Midnight	Deadline	Submit topic + SPEC draft
Day 6 — 09:00 (M1)	Canvas check + SPEC draft review	
Day 6 — 11:00 (M2)	“Show 1 thứ chạy được”	
Day 6 — 13:00 (M3)	SPEC final + demo flow draft	
Day 6 — 15:30 (M4)	Demo prep done + dry run	
Day 6 — 16:00 (M5)	Gallery walk + demo round	Prototype deadline
Day 6 — 17:00 (M6)	Top teams present + trao giải + closing	

Scoring: 60% nhóm / 40% cá nhân

Hạng mục	Loại	Điểm
SPEC	Nhóm	20
Prototype	Nhóm	15
Demo Day	Nhóm	15
Bài tập UX	Cá nhân	10
Tham gia checkpoint	Cá nhân	15
Bài tự nhận xét	Cá nhân	15
Tổng		100

4 tracks

Track	Lĩnh vực	Gợi ý
A	VinFast	AI cho ô tô / di chuyển / hậu mãi
B	Vinmec	AI cho y tế / trải nghiệm bệnh nhân
C	VinUni – VinSchool	AI cho giáo dục / vận hành trường
D	Tự chọn	Tự chọn lĩnh vực, phải có user thật

SPEC deliverable — 6 items

- AI Product Canvas (Value / Trust / Feasibility + Learning Signal)
- User Stories × 4 paths (Happy / Low-confidence / Failure / Correction)
- Eval metrics: 3 metrics + threshold + red flag
- Top 3 failure modes (Trigger / Hậu quả / Mitigation)
- ROI 3 kịch bản (conservative / realistic / optimistic)
- Mini AI spec 1 trang

Deadline: midnight Day 5. Nộp trễ = trừ điểm.

Prototype deliverable — 3 options

Lựa chọn A: Demo bằng prompt

- Dùng ChatGPT / Claude
- System prompt + ví dụ đầu vào/đầu ra
- Quay video demo

Lựa chọn B: Mock flow

- Figma / slides mock UI
- Hiện thị 4 đường (đúng → sai)
- Prototype click được

Lựa chọn C: Code chạy thật

- Streamlit / Gradio / web app
- Tích hợp API thật
- Triển khai được

Vòng demo — quy định

1. Mỗi nhóm demo tại bàn mình (laptop + poster/slides)
2. 2–3 người ở lại trình bày, 2–3 người đi xem nhóm khác
3. Feedback đủ các team trong zone — dùng feedback form (3 tiêu chí × 1-5)
4. Nộp đủ feedback forms — chấm cả chất lượng review

Vibe-coding warning: Demo đẹp nhưng không có SPEC / logic = 0 điểm SPEC. SPEC quan trọng hơn code.

Chia nhóm + chọn track

Tạo nhóm 3–5 người

Chọn 1 trong 4 tracks

Bắt đầu brainstorm bài toán

16:00 → bắt đầu ngay

Midnight deadline — hôm nay

Submit trước midnight:

- Topic chốt — problem statement 1 câu
- SPEC draft — Canvas + hướng đi chính
- Phân công nhóm ai làm gì

Không cần hoàn chỉnh — draft đủ để GV review sáng Day 6.

